dVRK Clutch Mechanism Quickstart

Do everything once per arm. For a more detailed guide, see the detailed instructions.

Conventions. Most scripts have a `side = RIGHT/LEFT` hyperparameter near the top - set it before every run. Many also have a True/False toggle (`collect_data`, `overwrite`, `RESET_LABELS`) for switching between modes. Confirm the camera index in each script matches the camera for the current side (`v4l2-ctl --list-devices`).

1 Camera Calibration

Directory: `camera_calibration/`

1. Run `camera_test.py`. Twist the lens to maximize focus.
2. Run `take_pictures_for_calibration.py`. Press SPACE to capture, Q to quit. Take at least 50 images of the ChArUco target from varied angles, as close as possible while keeping it in frame.
3. Optional sanity check: run `calibrate_camera_debug.py` to verify corner detection.
4. Set `side` in `calibrate_camera.py`. Run it. Intrinsics are written to four locations automatically.

2 Hand-Eye Calibration

Directory: `hand_eye_calibration/`

1. Insert camera into `camera_mount.STL` (JST connectors up). Tape mount to arm with bottom edge ∼16 cm from MTM start, perpendicular to the arm. Tape down JST wires.
2. Tape asymmetric-circles target on MTM near its stationary point. Use a phone flashlight below the target to compensate for the IR filter.
3. Verify `camera_intrinsics.json` is for the correct side.
4. Set `side` and `collect_data` = True in `mtm_registration.py`. Confirm MTM is on. Run.
5. Capture 10–15 poses (head-on, twisted L/R, varied) with ENTER. ESC to finish.
6. Inspect output: blue (Z) axis along MTM, point at the stationary point. If not, repeat. Expect several iterations.
7. Once good, set `collect_data` = False to review or run `mtm_registration_debug.py` to debug.

3 LED Data Collection

Directory: `hand_eye_calibration/`

1. Assemble clutch: spring in bottom compartment, batteries (+ up), slide switch, LEDs on opposite sides. See video for how to set this up by clicking on the video walkthrough.
2. Attach clutch to MTM.
3. Set `side` in `led_data_collection.py`. Run.
4. Manipulate the MTM through varied poses, **keeping the clutch engaged the entire time**. ESC when done.

4 LED Triangulation and ROI Calibration

Directory: `led_calibration/`

1. Confirm `camera_intrinsics.json` and `link3_wrt_camera.txt` are present (written by earlier steps).
2. Set `side` in `label_leds.py`. Run.
3. For each frame: left-click LEFT LED, then RIGHT LED (order matters). Right-click to undo. `x` to skip, `q` to quit.
4. Validate: set `side` in `live_roi_view.py` and run. ESC quits, `r` toggles ROI. Verify ROI tracks the LEDs.

5 Live Operation

Directory: `live_scripts/`

1. Verify the camera index in each `clutch_signal_*.py` matches the correct device for that arm.
2. slide the IR filter in front of the camera on the camera mount
3. Run `clutch_signal_left.py` and/or `clutch_signal_right.py` (each is hardcoded to its arm; no `side` flag).
4. Subscribe downstream code to `led_left_on` and `led_right_on` (`std_msgs/Bool`, ∼100 Hz).

6 Recalibrating LEDs Only

Directory: `recalibrate_leds/`

Same scripts as before, but with side-tagged files (`led_data_{side}/`, `led_calibration_{side}.json`, `common_{side}.py`). Each script has its own `side` hyperparameter. `label_leds.py` writes to both `recalibrate_leds/` and `live_scripts/` - the second write is what makes the new calibration take effect. To redo hand-eye, restart the full setup.